

Object-Oriented Programming (OOPs) using Java

Lecture Note-2: Inheritance, Polymorphism, and Inner Classes

Target Environment: Java SE 21 (Long-Term Support)

Prepared by: Mr. Ronak Sir and Team

July 25, 2026

1 Inheritance: The Foundation of Code Reusability

Inheritance is a core pillar of Object-Oriented Programming that enables a class to acquire properties (fields) and behaviors (methods) from another class. It promotes code reuse, establishes hierarchical relationships, and enables polymorphic behavior.

1.1 Core Terminology

Table 1: Inheritance Terminology

Term	Definition
Superclass	The parent class from which properties are inherited (also called Base Class or Parent Class)
Subclass	The child class that inherits from the superclass (also called Derived Class or Child Class)
extends	The keyword used to establish inheritance relationship
super	A reference variable used to refer to immediate parent class object

1.2 Basic Inheritance Syntax

Listing 1: Basic Inheritance Implementation

```
1 // Superclass (Parent)
2 class Animal {
3     String name;
4     int age;
5
6     void eat() {
7         System.out.println(name + " is eating...");
8     }
9
10    void sleep() {
11        System.out.println(name + " is sleeping...");
12    }
13 }
14
15 // Subclass (Child) - Inherits from Animal
```

```

16 class Dog extends Animal {
17     String breed;
18
19     void bark() {
20         System.out.println(name + " is barking!");
21     }
22
23     void wagTail() {
24         System.out.println(name + " is wagging its tail.");
25     }
26 }
27
28 public class InheritanceDemo {
29     public static void main(String[] args) {
30         Dog myDog = new Dog();
31
32         // Inherited properties from Animal
33         myDog.name = "Buddy";
34         myDog.age = 3;
35         myDog.eat();           // From Animal class
36         myDog.sleep();        // From Animal class
37
38         // Dog's own properties
39         myDog.breed = "Golden Retriever";
40         myDog.bark();          // From Dog class
41         myDog.wagTail();       // From Dog class
42     }
43 }

```

Key Insight

The Dog class automatically receives all non-private members (fields and methods) of the Animal class. This means we don't have to rewrite the `eat()` and `sleep()` methods for every animal type.

2 Types of Inheritance in Java

Java supports different forms of inheritance relationships, but with important restrictions to maintain design simplicity and avoid ambiguity.

2.1 Classification Overview

Table 2: Types of Inheritance Supported in Java

Type	Description
Single Inheritance	A class inherits from exactly one superclass. This is the most common form in Java.
Multilevel Inheritance	A class inherits from a subclass, which itself inherits from another superclass (Grandparent $\rightarrow$ Parent $\rightarrow$ Child).
Hierarchical Inheritance	Multiple subclasses inherit from a single superclass. (One parent, many children)
Multiple Inheritance	A class inherits from more than one superclass. NOT SUPPORTED in Java via classes to avoid the “Diamond Problem”.
Hybrid Inheritance	Combination of multiple and hierarchical inheritance. NOT SUPPORTED directly in Java.

2.2 Visual Representation of Inheritance Types

Single Inheritance

Class A $\rightarrow$ Class B

Multilevel Inheritance

Class A $\rightarrow$ Class B $\rightarrow$ Class C

Hierarchical Inheritance

Class B
/ \
Class A $\rightarrow$ Class C Class D

Multiple Inheritance (NOT SUPPORTED)

Class B Class C
 \
 Class A

2.3 Implementation Examples

2.3.1 Single Inheritance

Listing 2: Single Inheritance Example

```
1 class Vehicle {
2     String brand;
3     int speed;
4
5     void start() {
6         System.out.println("Vehicle is starting...");
7     }
8 }
9
10 class Car extends Vehicle {
11     int doors;
12
13     void honk() {
14         System.out.println("Car is honking!");
15     }
16 }
```

2.3.2 Multilevel Inheritance

Listing 3: Multilevel Inheritance Example

```
1 class GrandParent {
2     void showGrandParent() {
3         System.out.println("GrandParent class method");
4     }
5 }
6
7 class Parent extends GrandParent {
8     void showParent() {
9         System.out.println("Parent class method");
10    }
11 }
12
13 class Child extends Parent {
14     void showChild() {
15         System.out.println("Child class method");
16    }
17 }
18
19 public class MultiLevelDemo {
20     public static void main(String[] args) {
21         Child obj = new Child();
22         obj.showGrandParent(); // Inherited from GrandParent
23         obj.showParent();      // Inherited from Parent
24         obj.showChild();       // Own method
25     }
26 }
```

2.3.3 Hierarchical Inheritance

Listing 4: Hierarchical Inheritance Example

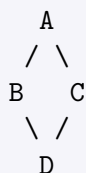
```
1 class Shape {
2     String color;
3
4     void displayColor() {
5         System.out.println("Color: " + color);
6     }
7 }
8
9 class Circle extends Shape {
10     double radius;
11
12     double calculateArea() {
13         return Math.PI * radius * radius;
14     }
15 }
16
17 class Rectangle extends Shape {
18     double length;
19     double width;
20
21     double calculateArea() {
22         return length * width;
23     }
24 }
25
26 class Triangle extends Shape {
27     double base;
28     double height;
29
30     double calculateArea() {
31         return 0.5 * base * height;
32     }
33 }
```

2.4 The Diamond Problem and Why Java Avoids Multiple Inheritance

The Diamond Problem Explained

When a class inherits from two superclasses that have a common ancestor, ambiguity arises regarding which superclass's method should be inherited.

Example:



If both B and C override a method from A, which version should D inherit? This creates a “diamond” of ambiguity. Java avoids this by not allowing multiple inheritance through classes.

2.5 Interfaces: The Solution for Multiple Inheritance

While Java doesn't support multiple inheritance through classes, it provides **Interfaces** as an alternative. A class can implement multiple interfaces, achieving the benefits of multiple inheritance without the diamond problem.

Listing 5: Multiple Inheritance through Interfaces

```
1 interface Flyable {
2     void fly();
3 }
4
5 interface Swimmable {
6     void swim();
7 }
8
9 class Duck implements Flyable, Swimmable {
10     @Override
11     public void fly() {
12         System.out.println("Duck is flying");
13     }
14
15     @Override
16     public void swim() {
17         System.out.println("Duck is swimming");
18     }
19 }
```

3 Method Overriding: Runtime Polymorphism

Method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its superclass. This is a fundamental mechanism for achieving runtime polymorphism.

3.1 Key Rules for Method Overriding

Table 3: Method Overriding Rules

Rule	Description
Same Signature	Method name and parameter list must be identical to the superclass method.
Return Type	Can be same or covariant (subtype of the original) (Covariant return types supported from Java 5)
Access Modifier	Cannot be more restrictive than the superclass method. (Can be more permissive)
Exception Throws	Cannot throw broader checked exceptions than the superclass method.
Static Methods	Cannot be overridden (they are hidden, not overridden).
Final Methods	Cannot be overridden.
Private Methods	Cannot be overridden (not visible to subclasses).
@Override Annotation	Recommended to explicitly indicate overriding.

3.2 Method Overriding Example

Listing 6: Method Overriding Implementation

```
1 class Animal {
2     void makeSound() {
3         System.out.println("Animal makes some sound");
4     }
5
6     void move() {
7         System.out.println("Animal moves");
8     }
9 }
10
11 class Dog extends Animal {
12     @Override
13     void makeSound() {
14         System.out.println("Dog barks: Woof! Woof!");
15     }
16
17     // Overriding with covariant return type (not shown here)
18     // but possible with more specific return types
19 }
20
```

```

21 class Cat extends Animal {
22     @Override
23     void makeSound() {
24         System.out.println("Cat meows: Meow! Meow!");
25     }
26 }
27
28 class Cow extends Animal {
29     @Override
30     void makeSound() {
31         System.out.println("Cow moos: Moo! Moo!");
32     }
33 }
34
35 public class OverridingDemo {
36     public static void main(String[] args) {
37         Animal animal1 = new Dog();
38         Animal animal2 = new Cat();
39         Animal animal3 = new Cow();
40
41         animal1.makeSound(); // Output: Dog barks: Woof! Woof!
42         animal2.makeSound(); // Output: Cat meows: Meow! Meow!
43         animal3.makeSound(); // Output: Cow moos: Moo! Moo!
44     }
45 }

```

3.3 Using the super Keyword in Overriding

The `super` keyword allows a subclass to access the superclass's version of an overridden method.

Listing 7: Using `super` with Overridden Methods

```

1 class Employee {
2     double salary = 50000;
3
4     void displaySalary() {
5         System.out.println("Base salary: Rs. " + salary);
6     }
7 }
8
9 class Manager extends Employee {
10     double bonus = 20000;
11     double totalSalary;
12
13     @Override
14     void displaySalary() {
15         // Calling superclass method using super
16         super.displaySalary();
17
18         totalSalary = salary + bonus;
19         System.out.println("Bonus: Rs. " + bonus);
20         System.out.println("Total salary: Rs. " + totalSalary);
21     }
22 }
23
24 public class SuperDemo {
25     public static void main(String[] args) {
26         Manager mgr = new Manager();

```

```

27         mgr.displaySalary();
28     }
29 }

```

3.4 Method Overriding vs Method Overloading

Table 4: Overriding vs Overloading Comparison

Feature	Method Overriding	Method Overloading
Relationship	Between superclass and sub-class	Within the same class
Parameter List	Must be identical	Must be different
Return Type	Same or covariant	Can be different
Access Modifier	Cannot be more restrictive	No restriction
Exception Throws	Cannot be broader	No restriction
Binding	Runtime (Dynamic)	Compile-time (Static)
Keywords	@Override, super	None specific

4 Dynamic Method Dispatch: Runtime Polymorphism

Dynamic Method Dispatch (DMD) is the mechanism by which a call to an overridden method is resolved at runtime rather than compile-time. This is the foundation of runtime polymorphism in Java.

4.1 The Concept Explained

When a superclass reference variable refers to a subclass object, and a method is called on that reference, the JVM determines which method implementation to execute based on the actual object type, not the reference type.

Core Principle

“What code gets executed depends on the object’s actual type, not the reference variable’s type.”

4.2 Dynamic Method Dispatch Example

Listing 8: Dynamic Method Dispatch Demonstration

```
1 class Shape {
2     void draw() {
3         System.out.println("Drawing a shape");
4     }
5
6     void calculateArea() {
7         System.out.println("Calculating area of shape");
8     }
9 }
10
11 class Circle extends Shape {
12     @Override
13     void draw() {
14         System.out.println("Drawing a circle");
15     }
16
17     @Override
18     void calculateArea() {
19         System.out.println("Area =   r   ");
20     }
21 }
22
23 class Square extends Shape {
24     @Override
25     void draw() {
26         System.out.println("Drawing a square");
27     }
28
29     @Override
30     void calculateArea() {
31         System.out.println("Area = side ");
32     }
33 }
34
35 public class DMDDemo {
36     public static void main(String[] args) {
```

```

37      // Superclass reference pointing to subclass objects
38      Shape shape1 = new Circle();
39      Shape shape2 = new Square();
40      Shape shape3 = new Shape();
41
42      // Method call resolved at runtime based on actual object
43      shape1.draw();           // Output: Drawing a circle
44      shape2.draw();           // Output: Drawing a square
45      shape3.draw();           // Output: Drawing a shape
46
47      shape1.calculateArea(); // Output: Area = r
48      shape2.calculateArea(); // Output: Area = side
49      shape3.calculateArea(); // Output: Calculating area of shape
50  }
51 }

```

4.3 Real-World Application: Polymorphic Arrays

Listing 9: Polymorphic Array Processing

```

1  class Employee {
2      String name;
3      double baseSalary;
4
5      Employee(String name, double baseSalary) {
6          this.name = name;
7          this.baseSalary = baseSalary;
8      }
9
10     double calculateSalary() {
11         return baseSalary;
12     }
13
14     void displayInfo() {
15         System.out.println(name + " - Salary: Rs. " + calculateSalary()
16             );
17     }
18 }
19
20 class Manager extends Employee {
21     double bonus;
22
23     Manager(String name, double baseSalary, double bonus) {
24         super(name, baseSalary);
25         this.bonus = bonus;
26     }
27
28     @Override
29     double calculateSalary() {
30         return super.calculateSalary() + bonus;
31     }
32 }
33
34 class Developer extends Employee {
35     double projectAllowance;

```

```

36     Developer(String name, double baseSalary, double projectAllowance)
37     {
38         super(name, baseSalary);
39         this.projectAllowance = projectAllowance;
40     }
41
42     @Override
43     double calculateSalary() {
44         return super.calculateSalary() + projectAllowance;
45     }
46
47     class Intern extends Employee {
48         double stipendBonus;
49
50         Intern(String name, double baseSalary, double stipendBonus) {
51             super(name, baseSalary);
52             this.stipendBonus = stipendBonus;
53         }
54
55         @Override
56         double calculateSalary() {
57             return super.calculateSalary() + stipendBonus;
58         }
59     }
60
61     public class PolymorphicArrayDemo {
62         public static void main(String[] args) {
63             // Array of superclass references
64             Employee[] employees = new Employee[4];
65
66             employees[0] = new Employee("John", 50000);
67             employees[1] = new Manager("Sarah", 80000, 20000);
68             employees[2] = new Developer("Mike", 60000, 15000);
69             employees[3] = new Intern("Emma", 30000, 5000);
70
71             // Polymorphic processing - each object calculates its own
72             salary
73             for (Employee emp : employees) {
74                 emp.displayInfo();
75             }
76         }
77     }

```

4.4 The instanceof Operator

The `instanceof` operator is used to check whether an object is an instance of a specific class or implements a specific interface.

Listing 10: Using instanceof Operator

```

1 public class InstanceOfDemo {
2     public static void main(String[] args) {
3         Employee emp1 = new Manager("Alice", 90000, 25000);
4         Employee emp2 = new Developer("Bob", 70000, 18000);
5
6         // Checking object types at runtime
7         if (emp1 instanceof Manager) {

```

```

8         System.out.println("emp1 is a Manager");
9         Manager mgr = (Manager) emp1; // Type casting
10        System.out.println("Bonus: Rs. " + mgr.bonus);
11    }
12
13    if (emp2 instanceof Manager) {
14        System.out.println("emp2 is a Manager");
15    } else {
16        System.out.println("emp2 is not a Manager");
17    }
18
19    // Checking multiple types
20    if (emp1 instanceof Employee) {
21        System.out.println("emp1 is an Employee");
22    }
23 }
24 }

```

4.5 Upcasting and Downcasting

Type Casting in Inheritance

Upcasting: Converting a subclass reference to a superclass reference (Implicit, safe).

```

1 Employee emp = new Manager("John", 80000, 20000); // Upcasting

```

Downcasting: Converting a superclass reference to a subclass reference (Explicit, requires instanceof check).

```

1 if (emp instanceof Manager) {
2     Manager mgr = (Manager) emp; // Downcasting
3 }

```

5 Inner Classes (Nested Classes)

Java allows defining a class within another class. These are called nested classes or inner classes. They are a way to logically group classes that are only used in one place, increase encapsulation, and can lead to more readable and maintainable code.

5.1 Types of Inner Classes

Table 5: Types of Inner Classes in Java

Type	Description
Static Nested Class	A static class defined within another class. Can access only static members of the outer class.
Non-static Inner Class	A non-static class defined within another class. Has access to all members of the outer class.
Local Inner Class	A class defined inside a method or block. Scope is limited to that method.
Anonymous Inner Class	A class defined without a name, usually for implementing interfaces or extending classes.

5.2 Static Nested Class

A static nested class is a class that is defined as `static` within another class. It behaves like a top-level class but is nested for packaging convenience.

Listing 11: Static Nested Class Example

```
1 class OuterClass {
2     private static String staticMessage = "Static Message";
3     private String instanceMessage = "Instance Message";
4
5     // Static nested class
6     static class StaticNestedClass {
7         void display() {
8             // Can access only static members of outer class
9             System.out.println("Static Nested Class");
10            System.out.println("Accessing static member: " +
11                staticMessage);
12
13            // Cannot access non-static members
14            // System.out.println(instanceMessage); // ERROR!
15        }
16
17        static void staticDisplay() {
18            System.out.println("Static method in nested class");
19        }
20    }
21 }
22 public class StaticNestedDemo {
```

```

23     public static void main(String[] args) {
24         // Creating instance of static nested class
25         OuterClass.StaticNestedClass nested = new OuterClass.
                StaticNestedClass();
26         nested.display();
27
28         // Calling static method of nested class
29         OuterClass.StaticNestedClass.staticDisplay();
30     }
31 }

```

5.3 Non-static Inner Class (Member Inner Class)

A non-static inner class is associated with an instance of the outer class and has access to all members (including private ones) of the outer class.

Listing 12: Non-static Inner Class Example

```

1  class OuterClass {
2      private String message = "Hello from Outer";
3      private static String staticMessage = "Static Message";
4
5      // Non-static inner class
6      class InnerClass {
7          void display() {
8              // Can access all members of outer class
9              System.out.println("Inner Class");
10             System.out.println("Accessing private member: " + message);
11             System.out.println("Accessing static member: " +
                    staticMessage);
12         }
13
14         void modifyOuter() {
15             // Can modify outer class members
16             message = "Modified by Inner";
17         }
18     }
19 }
20
21 public class InnerClassDemo {
22     public static void main(String[] args) {
23         // Must create outer class instance first
24         OuterClass outer = new OuterClass();
25
26         // Then create inner class instance using outer instance
27         OuterClass.InnerClass inner = outer.new InnerClass();
28         inner.display();
29
30         // Demonstrate modification
31         inner.modifyOuter();
32         // Accessing outer directly to see change
33         // (Not directly accessible if private, but demonstrates
            concept)
34
35         // Alternative one-line creation
36         OuterClass.InnerClass inner2 = new OuterClass().new InnerClass
            ();
37         inner2.display();

```

```

38     }
39 }

```

5.4 Local Inner Class

A local inner class is defined inside a method or a block. Its scope is limited to that method.

Listing 13: Local Inner Class Example

```

1  class OuterClass {
2      void outerMethod() {
3          final String localVariable = "Local variable";
4          String effectivelyFinal = "Effectively final"; // Not modified
5              after
6
7          // Local inner class
8          class LocalInnerClass {
9              void display() {
10                 // Can access local variables if they are final or
11                 effectively final
12                 System.out.println("Local Inner Class");
13                 System.out.println("Accessing local variable: " +
14                     localVariable);
15                 System.out.println("Accessing effectively final: " +
16                     effectivelyFinal);
17
18                 // Cannot modify local variables
19                 // localVariable = "Modified"; // ERROR!
20             }
21         }
22
23         // Creating instance of local inner class
24         LocalInnerClass localInner = new LocalInnerClass();
25         localInner.display();
26     }
27
28     void anotherMethod() {
29         // Cannot access LocalInnerClass here - out of scope
30         // LocalInnerClass local = new LocalInnerClass(); // ERROR!
31     }
32 }
33
34 public class LocalInnerDemo {
35     public static void main(String[] args) {
36         OuterClass outer = new OuterClass();
37         outer.outerMethod();
38     }
39 }

```

5.5 Anonymous Inner Class

An anonymous inner class is a class without a name that is created and instantiated at the same time. It is commonly used for implementing interfaces or extending classes in a single expression.

Listing 14: Anonymous Inner Class Examples

```

1  // Interface for demonstration
2  interface Greeting {

```

```

3     void greet();
4 }
5
6 // Abstract class for demonstration
7 abstract class Animal {
8     abstract void makeSound();
9 }
10
11 public class AnonymousInnerDemo {
12     public static void main(String[] args) {
13         // Anonymous inner class implementing an interface
14         Greeting greeting = new Greeting() {
15             @Override
16             public void greet() {
17                 System.out.println("Hello from anonymous inner class!");
18             }
19         };
20         greeting.greet();
21
22         // Anonymous inner class extending an abstract class
23         Animal dog = new Animal() {
24             @Override
25             void makeSound() {
26                 System.out.println("Woof! Woof!");
27             }
28
29             // Additional methods can be added but cannot be accessed
30             // through the reference type
31             void wagTail() {
32                 System.out.println("Wagging tail");
33             }
34         };
35         dog.makeSound();
36         // dog.wagTail(); // ERROR! Not available in Animal reference
37
38         // Anonymous inner class with constructor arguments
39         Thread thread = new Thread(new Runnable() {
40             @Override
41             public void run() {
42                 System.out.println("Running in anonymous thread");
43             }
44         });
45         thread.start();
46
47         // Using lambda expression (Java 8+) - alternative to anonymous
48         class
49         Runnable lambdaRunnable = () -> System.out.println("Using
50             lambda");
51         lambdaRunnable.run();
52     }
53 }

```

5.6 Accessing Outer Class Members

Listing 15: Accessing Outer Class Elements from Inner Class

```

1  class OuterClass {
2      private String message = "Outer message";
3      private static String staticMessage = "Static message";
4
5      class InnerClass {
6          private String message = "Inner message"; // Shadowing outer
7              message
8
9          void display() {
10              // Accessing inner class's own member
11              System.out.println("Inner message: " + this.message);
12
13              // Accessing outer class's instance member
14              System.out.println("Outer message: " + OuterClass.this.
15                  message);
16
17              // Accessing outer class's static member
18              System.out.println("Outer static: " + OuterClass.
19                  staticMessage);
20          }
21      }
22
23      static class StaticNestedClass {
24          void display() {
25              // Cannot access non-static members of outer class
26              // System.out.println(message); // ERROR!
27
28              // Can access static members
29              System.out.println("Static message: " + staticMessage);
30
31              // Can access static methods
32              System.out.println("Static method: " + staticMethod());
33          }
34      }
35
36      static String staticMethod() {
37          return "Static method called";
38      }
39  }
40
41  public class AccessOuterDemo {
42      public static void main(String[] args) {
43          OuterClass outer = new OuterClass();
44          OuterClass.InnerClass inner = outer.new InnerClass();
45          inner.display();
46
47          OuterClass.StaticNestedClass staticNested = new OuterClass.
48              StaticNestedClass();
49          staticNested.display();
50      }
51  }

```

5.7 Why Use Inner Classes?

- **Logical Grouping:** Classes that are only used by one class can be nested within it.
- **Increased Encapsulation:** Inner classes can access private members of the outer class.

- **More Readable Code:** Related classes are kept together.
- **Callbacks and Event Handling:** Anonymous inner classes are commonly used in GUI programming for event handlers.
- **Closure-like Behavior:** Inner classes can capture the state of the enclosing method (local inner classes).

6 Comprehensive Integration Examples

6.1 Example 1: Vehicle Hierarchy with Polymorphism

Listing 16: Complete Vehicle Management System

```
1 // Base class
2 abstract class Vehicle {
3     private String brand;
4     private String model;
5     private int year;
6
7     public Vehicle(String brand, String model, int year) {
8         this.brand = brand;
9         this.model = model;
10        this.year = year;
11    }
12
13    // Abstract method - must be implemented by subclasses
14    public abstract void startEngine();
15
16    // Concrete method - can be overridden
17    public void displayInfo() {
18        System.out.println("Brand: " + brand);
19        System.out.println("Model: " + model);
20        System.out.println("Year: " + year);
21    }
22
23    public String getBrand() { return brand; }
24    public String getModel() { return model; }
25    public int getYear() { return year; }
26 }
27
28 // Car subclass
29 class Car extends Vehicle {
30     private int doors;
31     private boolean isElectric;
32
33     public Car(String brand, String model, int year, int doors, boolean
34         isElectric) {
35         super(brand, model, year);
36         this.doors = doors;
37         this.isElectric = isElectric;
38     }
39
40     @Override
41     public void startEngine() {
42         System.out.println("Starting car engine...");
43         if (isElectric) {
44             System.out.println("Electric motor activated silently");
45         } else {
46             System.out.println("Internal combustion engine started");
47         }
48         System.out.println("Car is ready to drive!");
49     }
50
51     @Override
52     public void displayInfo() {
```

```

52         super.displayInfo();
53         System.out.println("Doors: " + doors);
54         System.out.println("Electric: " + (isElectric ? "Yes" : "No"));
55     }
56 }
57
58 // Motorcycle subclass
59 class Motorcycle extends Vehicle {
60     private boolean hasSidecar;
61     private int engineCC;
62
63     public Motorcycle(String brand, String model, int year, boolean
        hasSidecar, int engineCC) {
64         super(brand, model, year);
65         this.hasSidecar = hasSidecar;
66         this.engineCC = engineCC;
67     }
68
69     @Override
70     public void startEngine() {
71         System.out.println("Kicking motorcycle engine...");
72         System.out.println("Vroom Vroom!");
73         System.out.println("Motorcycle is ready to ride!");
74     }
75
76     @Override
77     public void displayInfo() {
78         super.displayInfo();
79         System.out.println("Engine CC: " + engineCC);
80         System.out.println("Sidecar: " + (hasSidecar ? "Yes" : "No"));
81     }
82 }
83
84 // Truck subclass
85 class Truck extends Vehicle {
86     private double payloadCapacity;
87     private int wheels;
88
89     public Truck(String brand, String model, int year, double
        payloadCapacity, int wheels) {
90         super(brand, model, year);
91         this.payloadCapacity = payloadCapacity;
92         this.wheels = wheels;
93     }
94
95     @Override
96     public void startEngine() {
97         System.out.println("Starting heavy duty truck engine...");
98         System.out.println("Engine roaring to life!");
99         System.out.println("Truck is ready for hauling!");
100    }
101
102    @Override
103    public void displayInfo() {
104        super.displayInfo();
105        System.out.println("Payload Capacity: " + payloadCapacity + "
            tons");
106        System.out.println("Wheels: " + wheels);

```

```

107     }
108 }
109
110 public class VehicleManagementSystem {
111     public static void main(String[] args) {
112         // Creating polymorphic array
113         Vehicle[] vehicles = new Vehicle[4];
114         vehicles[0] = new Car("Tesla", "Model 3", 2023, 4, true);
115         vehicles[1] = new Car("Honda", "Civic", 2022, 4, false);
116         vehicles[2] = new Motorcycle("Harley-Davidson", "Street 750",
117             2021, false, 750);
118         vehicles[3] = new Truck("Volvo", "FH16", 2023, 25.5, 6);
119
120         // Processing all vehicles polymorphically
121         System.out.println("=== Vehicle Showroom ===");
122         for (Vehicle v : vehicles) {
123             System.out.println("\n--- Vehicle Details ---");
124             v.displayInfo();
125             System.out.println("Starting Engine:");
126             v.startEngine();
127             System.out.println("-----");
128         }
129     }
130 }

```

6.2 Example 2: Banking System with Inner Classes

Listing 17: Banking System Using Inner Classes

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 class Bank {
5     private String bankName;
6     private String branch;
7     private List<Account> accounts;
8
9     // Static nested class for transaction record
10    static class Transaction {
11        private String type;
12        private double amount;
13        private String timestamp;
14
15        Transaction(String type, double amount, String timestamp) {
16            this.type = type;
17            this.amount = amount;
18            this.timestamp = timestamp;
19        }
20
21        void displayTransaction() {
22            System.out.println("    " + timestamp + ": " + type + " - Rs.
23                " + amount);
24        }
25    }
26
27    // Non-static inner class for Account
28    class Account {

```

```

28     private String accountNumber;
29     private String accountHolder;
30     private double balance;
31     private List<Transaction> transactions;
32
33     Account(String accountNumber, String accountHolder, double
        initialDeposit) {
34         this.accountNumber = accountNumber;
35         this.accountHolder = accountHolder;
36         this.balance = initialDeposit;
37         this.transactions = new ArrayList<>();
38         addTransaction("Initial Deposit", initialDeposit);
39     }
40
41     void deposit(double amount) {
42         if (amount > 0) {
43             balance += amount;
44             addTransaction("Deposit", amount);
45             System.out.println("Deposited Rs. " + amount + "
                successfully");
46         } else {
47             System.out.println("Invalid deposit amount");
48         }
49     }
50
51     void withdraw(double amount) {
52         if (amount > 0 && amount <= balance) {
53             balance -= amount;
54             addTransaction("Withdrawal", -amount);
55             System.out.println("Withdrawn Rs. " + amount + "
                successfully");
56         } else {
57             System.out.println("Insufficient balance or invalid
                amount");
58         }
59     }
60
61     private void addTransaction(String type, double amount) {
62         // Using Local inner class for timestamp formatting
63         class Timestamp {
64             String getCurrentTime() {
65                 return java.time.LocalDateTime.now().toString();
66             }
67         }
68         Timestamp ts = new Timestamp();
69         transactions.add(new Transaction(type, amount, ts.
            getCurrentTime()));
70     }
71
72     void printStatement() {
73         System.out.println("=== Bank Statement ===");
74         System.out.println("Bank: " + bankName);
75         System.out.println("Branch: " + branch);
76         System.out.println("Account: " + accountNumber);
77         System.out.println("Holder: " + accountHolder);
78         System.out.println("Current Balance: Rs. " + balance);
79         System.out.println("\nTransaction History:");
80         for (Transaction t : transactions) {

```

```

81         t.displayTransaction();
82     }
83     System.out.println("=====");
84 }
85 }
86
87 public Bank(String bankName, String branch) {
88     this.bankName = bankName;
89     this.branch = branch;
90     this.accounts = new ArrayList<>();
91 }
92
93 public Account createAccount(String accountNumber, String
94     accountHolder, double initialDeposit) {
95     Account account = new Account(accountNumber, accountHolder,
96         initialDeposit);
97     accounts.add(account);
98     System.out.println("Account created successfully!");
99     return account;
100 }
101
102 public Account findAccount(String accountNumber) {
103     for (Account acc : accounts) {
104         if (acc.accountNumber.equals(accountNumber)) {
105             return acc;
106         }
107     }
108     return null;
109 }
110
111 public void displayAllAccounts() {
112     System.out.println("=== All Accounts in " + bankName + " ===");
113     for (Account acc : accounts) {
114         System.out.println("Account: " + acc.accountHolder + " (" +
115             acc.accountNumber + ")");
116         System.out.println("Balance: Rs. " + acc.balance);
117         System.out.println("---");
118     }
119 }
120
121 }
122
123 public class BankingSystemDemo {
124     public static void main(String[] args) {
125         // Create bank
126         Bank bank = new Bank("National Bank", "Main Branch");
127
128         // Create accounts
129         Bank.Account account1 = bank.createAccount("ACC001", "Rahul
130             Sharma", 50000);
131         Bank.Account account2 = bank.createAccount("ACC002", "Priya
132             Patel", 75000);
133
134         // Perform transactions
135         account1.deposit(15000);
136         account1.withdraw(20000);
137         account2.deposit(10000);
138         account2.withdraw(30000);

```

```

134         // Print statements
135         account1.printStatement();
136         account2.printStatement();
137
138         // Display all accounts
139         bank.displayAllAccounts();
140     }
141 }

```

7 Summary and Key Takeaways

Key Concepts Summary

- **Inheritance:** Enables code reuse and establishes parent-child relationships using `extends`.
- **Types of Inheritance:** Single, Multilevel, and Hierarchical are supported. Multiple and Hybrid are not supported to avoid the Diamond Problem.
- **Method Overriding:** Provides runtime polymorphism by allowing subclasses to provide specific implementations of superclass methods.
- **Dynamic Method Dispatch:** The JVM determines which method to execute at runtime based on the actual object type, not the reference type.
- **Inner Classes:** Static nested, non-static inner, local, and anonymous inner classes provide ways to logically group classes and increase encapsulation.
- **Polymorphism:** Enables a single interface to represent different underlying forms, making code more flexible and maintainable.

Best Practices

1. Use `@Override` annotation when overriding methods to catch errors at compile time.
2. Prefer composition over inheritance when possible.
3. Use interfaces to achieve multiple inheritance of type.
4. Use inner classes only when the inner class is logically coupled with the outer class.
5. Avoid deep inheritance hierarchies (keep it to 2-3 levels).
6. Use the `instanceof` operator before downcasting to avoid `ClassCastException`.
7. Use anonymous inner classes sparingly; consider lambda expressions (Java 8+) for functional interfaces.